

Tutorial 2: MV Effects and Reference Voltage

Exercise Solutions

Before running the solutions

Open `Tutorial_2_MV_Effects_Reference_Voltage.ipynb` and, in a fresh Python kernel, run Sections 2.2–2.4. These sections load the arrays and define the model, scaling, validation and search functions used below. There is no need to run the tutorial's Sections 2.5–2.6 first. Paste the seven Python listings into the matching Exercise Workspace cells and run them in order. The listings are complete code, not pseudocode.

These solutions follow the current notebook's data convention: all 31 columns of `V.pk1` are targets; the three columns of `V_secondary.pk1` are additional supplied reference inputs. Thus the two models have 62 and 65 inputs, respectively, and the same 31 outputs. The precise physical extraction node of the secondary-reference file is not documented in the supplied data package. No customer column is removed without a verified reference-to-customer mapping.

Both exercises use ReLU, 40 epochs, three blocked folds, a learning rate of 10^{-3} , L2 strength 10^{-5} and batch size 72. This exercise budget is independent of the tutorial's quick/full mode. All selection uses training data only. Test data are evaluated only after choosing the configuration and seed. This is a classroom reuse of the tutorial's test period, not a new independent test set.

E1.1: Prepare the two input sets and common targets

Keep the P/Q columns from all 31 customers in both input sets. Append the three supplied reference columns only to the augmented model. Do not fit scalars here: validation must fit them separately inside each training fold.

E1.1

```
e1_reference_columns = REFERENCE_PHASES.copy()
e1_target_indices = np.arange(C, dtype=int)
e1_target_customers = e1_target_indices + 1
e1_model_inputs = {
    "PQ_only": PQ_mv_tr.astype(np.float32, copy=False),
    "PQ_plus_Vref": np.column_stack([
        PQ_mv_tr, V_secondary_mv_tr
    ]).astype(np.float32),
}
e1_targets = V_mv_tr[:, e1_target_indices].astype(
    np.float32, copy=False
)
e1_model_names = list(e1_model_inputs)

assert e1_reference_columns == [
    "phase_a", "phase_b", "phase_c"
]
assert e1_model_inputs["PQ_only"].shape == (6048, 62)
assert e1_model_inputs["PQ_plus_Vref"].shape == (6048, 65)
```

```

assert e1_targets.shape == (6048, 31)
assert np.array_equal(
    e1_model_inputs["PQ_plus_Vref"][:, 62:],
    V_secondary_mv_tr,
)

display(pd.DataFrame([
    {
        "Model": MODEL_LABELS[name],
        "Training rows": len(e1_targets),
        "Inputs": e1_model_inputs[name].shape[1],
        "Outputs": e1_targets.shape[1],
    }
    for name in e1_model_names
]))
print("Reference columns:", e1_reference_columns)
print("Target customers:", e1_target_customers.tolist())

# Keep exercise results separate from tutorial results.
e1_results_directory = (
    TUTORIAL_DIRECTORY / "results" / "exercises"
)
e1_results_directory.mkdir(parents=True, exist_ok=True)

```

E1.2: Compare 93 and 155 hidden neurons

Use the existing `run_grid_search()` function. It builds fresh models and fits scalers on each fold's training rows only. The two configurations require $2 \times 3 \times 2 = 12$ model fits.

E1.2

```

e1_epochs = 40
e1_seed = 42
e1_k = 3
e1_search_space = {
    "hidden_neurons": [93, 155],
    "activation": ["relu"],
    "lr": [1e-3],
    "weight_decay": [1e-5],
    "batch_size": [72],
    "epochs": [e1_epochs],
}
e1_results = run_grid_search(
    e1_model_inputs,
    e1_targets,
    e1_search_space,
    k=e1_k,
    seed=e1_seed,
)
e1_results.to_csv(
    e1_results_directory / "exercise_configurations.csv",
    index=False,
)

```

E1.3: Display validation scores and select a configuration

The joint score is the arithmetic mean of the two models' mean validation RMSE values. Select one shared configuration using its unrounded score. The reported standard deviations describe the

three fold scores (`numpy.std`, `ddof=0`), not confidence intervals.

E1.3

```
e1_results = e1_results.sort_values(
    "joint_RMSE_kfold", kind="stable"
).reset_index(drop=True)
e1_score_columns = [
    "hidden_neurons", "activation", "epochs",
    "PQ_only_RMSE_kfold", "PQ_only_RMSE_std",
    "PQ_plus_Vref_RMSE_kfold", "PQ_plus_Vref_RMSE_std",
    "joint_RMSE_kfold",
]
display(e1_results[e1_score_columns].round(6))

e1_best_row = e1_results.iloc[0]
e1_best_config = {
    "hidden_neurons": int(e1_best_row["hidden_neurons"]),
    "activation": str(e1_best_row["activation"]),
    "lr": float(e1_best_row["lr"]),
    "weight_decay": float(e1_best_row["weight_decay"]),
    "batch_size": int(e1_best_row["batch_size"]),
    "epochs": int(e1_best_row["epochs"]),
}
assert e1_best_config["activation"] == "relu"
assert e1_best_config["epochs"] == 40
print("Selected exercise configuration:", e1_best_config)
```

E2.1: Compare seeds 0, 1 and 2

Keep the configuration and folds fixed. This step uses training-period data only and requires $3 \times 3 \times 2 = 18$ additional model fits.

E2.1

```
e2_seeds = [0, 1, 2]
e2_seed_results = run_seed_search(
    e1_model_inputs,
    e1_targets,
    e1_best_config,
    e2_seeds,
    k=e1_k,
)
e2_seed_results = e2_seed_results.sort_values(
    "joint_RMSE_kfold", kind="stable"
).reset_index(drop=True)
display(e2_seed_results[[
    "seed",
    "PQ_only_RMSE_kfold", "PQ_only_RMSE_std",
    "PQ_plus_Vref_RMSE_kfold", "PQ_plus_Vref_RMSE_std",
    "joint_RMSE_kfold",
]].round(6))
e2_selected_seed = int(e2_seed_results.iloc[0]["seed"])
print("Selected exercise seed:", e2_selected_seed)
e2_seed_results.to_csv(
    e1_results_directory / "exercise_seeds.csv",
    index=False,
)
```

E2.2: Fit new scalers and train two new final models

Fit one common target scaler and one input scaler per model on all exercise training rows. Do not reuse the tutorial's final models or fold models. These two fits bring the exercise total to $12 + 18 + 2 = 32$.

E2.2

```
e2_output_scaler = MinMaxScaler().fit(e1_targets)
e2_y_train_scaled = e2_output_scaler.transform(
    e1_targets
).astype(np.float32)
e2_models = {}
e2_input_scalers = {}
e2_histories = {}
e2_training_rows = []

for name in e1_model_names:
    e2_input_scalers[name] = MinMaxScaler().fit(
        e1_model_inputs[name]
    )
    e2_x_train_scaled = e2_input_scalers[name].transform(
        e1_model_inputs[name]
    ).astype(np.float32)
    e2_models[name] = build_model(
        e2_x_train_scaled.shape[1],
        e2_y_train_scaled.shape[1],
        e1_best_config,
        seed=e2_selected_seed,
    )
    e2_histories[name] = train_model(
        e2_models[name],
        e2_x_train_scaled,
        e2_y_train_scaled,
        e1_best_config,
    )
    e2_completed_epochs = len(e2_histories[name].epoch)
    assert e2_completed_epochs == e1_best_config["epochs"]
    e2_training_rows.append({
        "Model": MODEL_LABELS[name],
        "Inputs": e2_models[name].input_shape[-1],
        "Outputs": e2_models[name].output_shape[-1],
        "Completed epochs": e2_completed_epochs,
    })

display(pd.DataFrame(e2_training_rows))
e2_run_settings = {
    "configuration": e1_best_config,
    "selected_seed": e2_selected_seed,
    "folds": e1_k,
    "training_fits": 32,
    "reference_source": "V_secondary.pkl",
    "reference_columns": e1_reference_columns,
    "target_customers": e1_target_customers.tolist(),
}
with (e1_results_directory / "exercise_settings.json").open(
    "w", encoding="utf-8"
) as settings_file:
    json.dump(e2_run_settings, settings_file, indent=2)
```

E2.3: Predict test voltages and report errors

Transform test inputs with the fitted training scalers. Never call `fit()` or `fit_transform()` on test data. The predictions are converted back to volts before calculating errors. For N time steps and C targets, the overall metrics are

$$\text{RMSE} = \sqrt{\frac{1}{NC} \sum_{t=1}^N \sum_{c=1}^C (\hat{V}_{t,c} - V_{t,c})^2}, \quad \text{MAE} = \frac{1}{NC} \sum_{t=1}^N \sum_{c=1}^C |\hat{V}_{t,c} - V_{t,c}|.$$

Per-customer RMSE averages over time only.

E2.3

```
e2_test_inputs = {
    "PQ_only": PQ_mv_te.astype(np.float32, copy=False),
    "PQ_plus_Vref": np.column_stack([
        PQ_mv_te, V_secondary_mv_te
    ]).astype(np.float32),
}
e2_test_targets = V_mv_te[:, e1_target_indices].astype(
    np.float32, copy=False
)
assert e2_test_inputs["PQ_only"].shape == (2016, 62)
assert e2_test_inputs["PQ_plus_Vref"].shape == (2016, 65)
assert e2_test_targets.shape == (2016, 31)

e2_predictions = {}
e2_errors = {}
e2_metric_rows = []
for name in e1_model_names:
    e2_x_test_scaled = e2_input_scalers[name].transform(
        e2_test_inputs[name]
    ).astype(np.float32)
    e2_predictions[name] = predict_unscaled(
        e2_models[name], e2_x_test_scaled, e2_output_scaler
    )
    assert e2_predictions[name].shape == e2_test_targets.shape
    assert np.isfinite(e2_predictions[name]).all()
    e2_errors[name] = e2_predictions[name] - e2_test_targets
    e2_metric_rows.append({
        "Model": MODEL_LABELS[name],
        "RMSE (V)": float(np.sqrt(np.mean(
            e2_errors[name] ** 2
        ))),
        "MAE (V)": float(np.mean(np.abs(e2_errors[name]))),
    })

e2_metrics = pd.DataFrame(e2_metric_rows)
display(e2_metrics.round(6))
e2_customer_rmse = pd.DataFrame({
    "Customer": e1_target_customers,
    "P/Q only RMSE (V)": np.sqrt(np.mean(
        e2_errors["PQ_only"] ** 2, axis=0
    )),
    "P/Q + Vref RMSE (V)": np.sqrt(np.mean(
        e2_errors["PQ_plus_Vref"] ** 2, axis=0
    )),
})
display(e2_customer_rmse.round(6))
e2_metrics.to_csv(
    e1_results_directory / "exercise_test_metrics.csv",
    index=False,
)
```

```
e2_customer_rmse.to_csv(
    e1_results_directory / "exercise_customer_rmse.csv",
    index=False,
)
```

E2.4: Inspect the worst P/Q-only customer

Choose the customer using the P/Q-only test RMSE, then use that same customer for both model comparisons. This is post-training inspection, not a further model-selection step. Residuals are estimate minus reference.

E2.4

```
e2_worst_position = int(np.argmax(
    e2_customer_rmse["P/Q only RMSE (V)"].to_numpy()
))
e2_worst_customer = int(
    e1_target_customers[e2_worst_position]
)
print("Worst P/Q-only customer:", e2_worst_customer)
display(e2_customer_rmse.iloc[[e2_worst_position]].round(6))

e2_test_hours = (
    np.arange(len(e2_test_targets)) * MINUTES_PER_STEP / 60
)
e2_fig, e2_axes = plt.subplots(
    2, 1, figsize=(11, 6), sharex=True
)
e2_axes[0].plot(
    e2_test_hours,
    e2_test_targets[:, e2_worst_position],
    color="black", label="Simulated reference",
)
for name, colour in [
    ("PQ_only", "tab:red"),
    ("PQ_plus_Vref", "tab:green"),
]:
    e2_axes[0].plot(
        e2_test_hours,
        e2_predictions[name][:, e2_worst_position],
        color=colour, label=MODEL_LABELS[name], alpha=0.8,
    )
    e2_axes[1].plot(
        e2_test_hours,
        e2_errors[name][:, e2_worst_position],
        color=colour, label=MODEL_LABELS[name], linewidth=0.8,
    )
e2_axes[0].set_ylabel("Voltage (V)")
e2_axes[0].set_title(
    f"Exercise: worst P/Q-only customer {e2_worst_customer}"
)
e2_axes[0].legend()
e2_axes[1].axhline(0, color="black", linestyle="--")
e2_axes[1].set_xlabel("Hours from start of test period")
e2_axes[1].set_ylabel("Estimate - reference (V)")
e2_axes[1].set_title("Voltage residuals")
e2_axes[1].legend()
e2_fig.tight_layout()
e2_fig.savefig(
    e1_results_directory / "exercise_worst_customer.png",
    dpi=160, bbox_inches="tight",
)
```

```
)
plt.show()
```

Verified execution example

The following results are from an actual run of the listings above, not the tutorial's tanh models. Values may vary slightly across environments. The code selects settings from validation scores; the numbers below must not be hard-coded as a replacement for that selection.

All 32 exercise fits completed without errors.

E1.2–E1.3: Configuration comparison

Mean and population standard deviation of fold RMSE, in volts:

Hidden	P/Q mean	P/Q std.	+Vref mean	+Vref std.	Joint
155	0.584702	0.002578	0.054287	0.000410	0.319494
93	0.593790	0.015297	0.063779	0.003495	0.328785

Selected hidden-neuron count: 155.

E2.1: Seed comparison

Seed	P/Q mean RMSE (V)	+Vref mean RMSE (V)	Joint (V)
1	0.597398	0.052285	0.324841
0	0.600967	0.052752	0.326860
2	0.607313	0.054578	0.330946

Selected seed: 1.

E2.2: Final training

Model	Inputs	Outputs	Completed epochs
P/Q only	62	31	40
P/Q + Vref	65	31	40

E2.3: Held-out test metrics

Model	RMSE (V)	MAE (V)
P/Q only	0.658955	0.479735
P/Q + Vref	0.047638	0.034860

Table 1: Test RMSE for each of the 31 target customers.

Customer	P/Q only RMSE (V)	P/Q + Vref RMSE (V)
1	0.564115	0.018393
2	0.709498	0.025437
3	0.648981	0.025153
4	0.572872	0.023709
5	0.752155	0.032014
6	0.660002	0.026363
7	0.564298	0.027801
8	0.736837	0.042775
9	0.652373	0.034059
10	0.576141	0.029734
11	0.739425	0.041125
12	0.652856	0.039666
13	0.567449	0.037427
14	0.746576	0.045619
15	0.648879	0.036716
16	0.574454	0.044865
17	0.735714	0.058042
18	0.654817	0.049783
19	0.588425	0.055577
20	0.744757	0.054893
21	0.662171	0.059403
22	0.579683	0.057289
23	0.750198	0.060591
24	0.670091	0.066627
25	0.584982	0.054939
26	0.751607	0.070531
27	0.665785	0.054701
28	0.563807	0.056967
29	0.745268	0.069582
30	0.669573	0.053266
31	0.580494	0.054017

E2.4: Worst P/Q-only customer

Customer 5: P/Q-only RMSE 0.752155 V; P/Q + Vref RMSE 0.032014 V.

Running E2.4 displays both requested panels and saves the plot here:

`results/exercises/exercise_worst_customer.png`

These are results under the current 31-target, supplied-reference convention. They do not establish the undocumented physical origin of the reference signals.